```
  "tags":["Python","Software Programming"]
}
```

Now that the Python book is indexed, we need to rerun the user's failed query. But this time, instead of running the query on the `tech_books` index, let's run it against the `tech_books_percolator` index. The query against the `percolator` index has the following special syntax.

**Listing 12.32 Searching for queries in the `percolator` index**

```
GET tech_books_percolator/_search
{
  "query": {
    "percolate": {
      "field": "query",
      "document": {
        "name":"Python in Action",
        "tags":["Python","Software Programming"]
      }
    }
  }
}
```

Specifies the percolate query

Sets the field's name to query

Specifies the document consisting of the original book indexed into tech_books

The `percolate` query expects two bits of input: a `field` with a value of `query` (this coincides with the property defined in the percolator mapping in listing 12.28) and a document, which is the same document we indexed in our `tech_books` index. All we need to do is check for any query that matches the given `document`. Fortunately, *Python in Action* is a match (we indexed a query into our `percolator` index earlier).

Given the Python document defined in listing 12.31, we can return a query from the `tech_books_percolator` index. This lets us inform the user that the book they were looking for is back in stock! Note that we can extended the query stored in the `tech_books_percolator` index with a specific user ID.

Percolators are a little tricky to understand, but once you understand the use case, they aren't difficult to implement. Keep in mind that there must always be an automated, semi-automated, or manual process to sync the operations a user performs against the data stored in the `percolator` index.

That's a wrap! By discussing advanced queries in this chapter, we've covered the search part of Elasticsearch. The final part is aggregating data, which is the next chapter's subject. Stay tuned to learn more about how we can find intelligence in our data by analyzing it with various mathematical and statistical functions.

## Summary

- Elasticsearch supports the `geo_point` and `geo_shape` data types to work with geodata.
- Geospatial queries fetch locations and addresses using coordinates formed from longitude and latitude values.